

The Program Is Where Ambiguity Stops

Local Closure and the Architecture of Natural-Language Programming

Watson Hartsoe

Working paper · 17 August 2026



Working Paper · AI-augmented research process
Evidence, prompts, and making history preserved.

Abstract

Natural-language programming is often described as deferred formalization: a user can begin with an incomplete request and introduce precision later as the interaction reveals what matters. The description is useful but still too weak. Earlier systems already distributed specification across world models, restricted languages, demonstrations, programmer knowledge, situation particulars, and repair. What is distinctive in contemporary language-model agents is not simply that formalization happens late. It is that different dimensions of a task can remain unsettled until a particular downstream transition requires them to become binding. I call that transition *local closure*. Reference can close when “that one” becomes a specific object; output shape can close in a schema while semantic correctness remains open; procedure can close in temporary code while the larger goal remains revisable; an action can be fully proposed yet remain uncommitted; a failure can close into a test; a rule can close into an active, revoked, or superseded temporal state. The stronger wager of this paper is that the operative program in natural-language systems is not the prompt, generated code, or runtime in isolation. It is the *topology of closure boundaries*: the ordered places where ambiguity is no longer permitted to pass unchanged into the next state transition. Reliability therefore does not require eliminating ambiguity globally. It requires consequence-proportional closure: keep interpretation revisable while mistakes are cheap and reversible, but force the relevant distinctions to settle before actions become difficult to undo. This also explains why some criteria should remain live human judgments rather than be prematurely compiled into metrics.

Keywords: natural-language programming; program specification; situated action; programming by demonstration; agentic systems; human-computer interaction; software semantics; commitment

1 The stronger question

A natural-language programming system can look radically new because a sentence appears to cross directly into consequence. Type a request; a file changes. Describe an operation; a tool runs. State a goal; a temporary program appears. Yet the historical record makes the obvious novelty claims difficult to defend. SHRDLU interpreted English against a symbolic world model and executed procedures in the early 1970s (Winograd 1973). Attempto Controlled English later translated a deliberately restricted English into formal representations and Prolog (Fuchs and Schwitter 1996). Pygmalion made concrete demonstration part of program construction decades before in-context examples became a routine interaction with language models (Smith 1993). Naur argued that competent programming depends on a theory that exceeds the program text itself (Naur 1985). Suchman and Clark with Wilkes-Gibbs showed, in different traditions, why successful action and reference cannot be reduced to self-sufficient prior expressions (Suchman 1985; Clark and Wilkes-Gibbs 1986).

The question, then, is not whether words can cause computation, whether a specification can be incomplete, or whether execution can teach the programmer something about the program. Those phenomena have precedents. A sharper question is: *when does a still-negotiable interpretation become binding for a particular consequence?*

The previous language of *restriction migration* helps locate formal constraint: in a grammar, a microworld, an example space, a schema, a tool contract, or a runtime. But location alone does not explain when a representation begins to govern an irreversible transition. A schema can be perfectly formal and still describe the wrong action. A deictic phrase can be linguistically underspecified

and still become perfectly sufficient once a referent has been established in a shared interaction. Formality and settlement are therefore not the same variable.

This paper calls the second variable *local closure*. A dimension is locally closed when one of its competing interpretations is made binding for the next operation, even though other dimensions of the larger task remain open. The claim is deliberately stronger than “formalization happens later.” It says that the operative architecture of natural-language programming is organized around multiple, differently placed closure events.

2 Formalization is not closure

Formalization makes distinctions explicit in a representation. Closure makes one distinction consequential for a state transition. The two often coincide, but they need not.

Structured output illustrates formalization without semantic closure. A JSON schema can make certain shapes impossible and guarantee that downstream code receives an object of an admitted form (OpenAI 2026b). But a structurally valid object can still identify the wrong target, encode an unsafe amount, or choose a semantically inappropriate operation. The representation has become stricter while the question “is this the right thing to do?” remains open.

Situated reference supplies the reverse pattern. Suchman treats intelligibility as dependent on particulars of the occasion rather than on a self-sufficient expression (Suchman 1985). Clark and Wilkes-Gibbs describe reference as a process in which a candidate expression can be accepted, expanded, repaired, or replaced until it becomes sufficient for the joint task (Clark and Wilkes-Gibbs 1986). The words themselves need not become more formally specified. A phrase such as “that one” can become locally closed because the interaction has reduced the viable referents to one actionable object.

Austin’s analysis of performative utterances sharpens the same distinction from another direction. The force of an utterance depends on procedures and circumstances, not only on sentence form (Austin 1962). In computational systems, the analogue is not that every tool call is a speech act. It is that an expression becomes consequential only inside an apparatus that supplies authority, admissible operations, state, and a path from representation to effect.

These cases motivate a compact distinction:

	Formalization	Local closure
Question	What has become explicit or machine-checkable?	What can no longer remain ambiguous before the next transition?
Typical object	grammar, schema, code, type, ledger	selected referent, accepted criterion, authorized action, active rule
Can occur alone?	yes: a valid structure can still be semantically wrong	yes: a situated choice can settle without a complete formal model
Failure signal	representation violates an admitted form	downstream action reveals that the wrong interpretation was allowed to pass

This distinction changes the design question. The goal is not maximum formalization. It is to

identify which ambiguity can remain productive and which ambiguity must be closed before it reaches a consequence that depends on the distinction.

3 Early systems close ambiguity upstream

SHRDLU and Attempto make useful historical contrasts because each buys tractability by closing large classes of ambiguity before runtime action, though at different sites.

Winograd’s blocks world sharply narrows the ontology, possible relations, available actions, and maintained state against which an English command can be interpreted (Winograd 1973). A command such as “put the red block on the blue one” is actionable only because the system already possesses a small universe in which the relevant entities, relations, preconditions, procedures, and effects are represented. Surface language can remain comparatively flexible because much of the world has already been closed.

Attempto makes another bargain visible. Its controlled English restricts the forms users may write so that the system can translate specifications through formal representations into executable or queryable structures (Fuchs and Schwitter 1996). Here, more of the ambiguity is closed in the admitted language itself.

Neither system is pure. SHRDLU also restricts language; Attempto also presupposes domain and interpreter machinery. The point is not to assign each system to one box. It is to notice that they make different closure sites dominant. What looks like “natural language becoming more executable” can often be restated more precisely as a change in what has already been settled before the user speaks.

4 Interactive traditions delay closure

Programming by demonstration complicates any story in which a complete rule must precede execution. Smith’s account of Pygmalion describes programming through concrete cases and manipulations rather than only through prior abstract description (Smith 1993). The abstraction can remain partially open while examples accumulate. What eventually becomes a reusable procedure is inferred or constructed through interaction with cases.

Naur’s theory-building argument deepens the point. Program text does not exhaust the knowledge required to understand why a program has its form or how to modify it when the world changes (Naur 1985). Formal semantics do not eliminate the programmer’s evolving theory. Some of what governs competent change remains outside the written program.

Suchman’s treatment of plans and repair introduces an even stronger limit on prior closure. Plans are resources for action, not exhaustive determinants of situated conduct (Suchman 1985). When an interaction breaks down, participants diagnose what kind of misunderstanding occurred and repair it. The operative specification therefore develops through a sequence of encounters between prior representation and present circumstances.

The lesson for contemporary prompting is not that everything is new again under another name. It is that postponing closure has a long lineage. The contemporary question is how many distinct

closure mechanisms can now coexist inside one task, and how dynamically the system can move among them.

5 The agent stack as a chain of closure operators

Contemporary language-model agents distribute settlement across representations that become binding at different times. Requirements-oriented prompt refinement explicitly treats the first user request as material for elicitation, analysis, specification, and validation rather than as finished program text (Shi et al. 2026). Natural-language harness work likewise places part of control policy in an editable linguistic representation interpreted by a runtime (Pan et al. 2026). These architectures make the initial prompt a beginning of control, not its complete location.

Other mechanisms close narrower dimensions. Structured outputs can close legal shape while leaving semantic choice open (OpenAI 2026b). Programmatic tool calling can close a bounded procedural region by synthesizing deterministic control flow for loops, branches, and tool orchestration, after which broader semantic judgment resumes (OpenAI 2026a). The important feature is not simply that code is generated. It is that procedure can become temporarily non-negotiable at one layer without forcing the whole task into code.

Action review closes another dimension: consequence. Ta and colleagues describe a reviewer that evaluates provisional tool calls before execution (Ta et al. 2026). This creates a state in which an action is semantically and structurally specific enough to be inspected, yet has not crossed into external effect. Serdar and Mertayak make this boundary itself a benchmark object: the relevant choice is whether to proceed or hold at a pre-commit action boundary (Serdar and Mertayak 2026). A crucial implication follows. Deciding what to do and committing the world to that decision are separable operations.

Failure and time add two further closure processes. Prompt-evolution research treats feedback from unsuccessful behavior as material for subsequent prompt change (Pei et al. 2026). A recurrent failure can therefore become a new invariant or test rather than remaining an informal memory of what went wrong. Zhu’s work on constraint influence shows the inverse problem: a revoked instruction can remain behaviorally active unless its changed validity is represented explicitly (Zhu 2026). A living specification needs not only additions but closure over temporal status: active, revoked, superseded, expired, or unresolved.

These mechanisms can be read as a family of local closure operators:

Closure dimension	What becomes binding	Typical mechanism
Referential	which object, span, file, or prior event is meant	interaction state, selection, clarification, repair
Structural	what forms a legal output may take	schema, type, constrained decoding
Procedural	which steps and branches will run in a bounded subtask	generated or handwritten control flow
Criterial	what counts as acceptable or failed	grader, test, invariant, human judgment
Consequential	whether a proposed operation crosses into external state change	review, authorization, commit gate
Temporal	which instructions currently bind	scoped state, revocation, supersession, tombstone

No single mechanism is the program. The program emerges from their ordering and interaction.

6 The program is the topology of closure boundaries

This leads to the paper’s central wager. The operative program in a natural-language system is best understood as the *topology of closure boundaries*: the arrangement of places where an underdetermined dimension becomes binding for the next state transition.

The word *topology* matters because closure is not merely a list of constraints. Order and adjacency change what the system can safely do. A schema before semantic review is different from a schema after an unsafe decision has already been made. A commit gate is useful only if it receives a representation rich enough to review. A tombstone matters only if the runtime consults active constraint state rather than raw historical text. A grader changes future behavior only if failures can propagate back into the governing specification. The same ingredients in another order form another program.

This view also explains why the prompt is the wrong unit. A prompt can initiate a task without containing its eventual referents, procedures, tests, permissions, or active constraint ledger. Generated code is also too small a unit because it may govern only a deterministic subregion. Runtime infrastructure is too broad if it ignores which constraints this particular interaction has accumulated. The operational object is the path by which provisional intention acquires successive settlements.

A useful reconstruction is therefore not:

PROMPT -> PROGRAM -> OUTPUT

but something closer to:

INTENTION -> OPEN INTERPRETATION -> LOCAL CLOSURES -> COMMIT -> CONSEQUENCE ->
REPAIR / REVISION

The closures need not all occur before execution. Some are generated by the consequences of prior attempts. The resulting specification is genealogical: it remembers what was settled, why a rule appeared, what failed, and what later ceased to bind.

7 Why not close everything?

If closure produces reliability, an obvious response is to close every dimension as early as possible. The field itself provides reasons not to.

First, some evaluative criteria are operational before they are fully articulable. A writer, designer, or musician can reject a candidate consistently without yet possessing a metric that captures the judgment. Converting such competence into a rubric may improve inspectability while also deleting distinctions the practitioner was actually using. This is not an argument for mysticism. It is a warning that explicit criteria and operative criteria can diverge.

Second, situated action cannot be reduced to a static representation of circumstances. Suchman's account makes the occasion itself part of intelligibility (Suchman 1985); Brooks's critique of representation emphasizes ongoing coupling to a changing environment rather than sole reliance on a central model (Brooks 1991). A larger context window may represent more of the world without becoming participation in the world. Systems that can re-inspect tools, files, sensors, or external state may therefore be safer than systems that merely carry a longer textual description of what used to be true.

Third, premature closure can remove productive resistance. A system that instantly supplies a plausible answer may allow an unresolved distinction to disappear before the user has discovered that it matters. In programming, this appears as a wrong abstraction stabilized too early. In writing, it can appear as fluent continuation that outruns judgment. The difficulty is to distinguish resistance that produces thought from friction that merely consumes effort.

The goal is therefore not maximal closure but *selective closure*: bind what must be stable for the next consequential transition, while allowing the rest of the problem to remain revisable.

8 Consequence-proportional closure

The preceding argument yields a design wager that can be tested. Semantic slack should generally decrease as the cost and irreversibility of a state transition increase.

A read-only search can tolerate an imprecise route if the user can inspect results cheaply. A draft patch can tolerate more unresolved interpretation than a deployed database migration. A proposed email can remain editable; a sent email cannot. A candidate purchase can remain negotiable; a transfer of funds should require stronger settlement of amount, recipient, authority, and intent. The relevant principle is not "ask for confirmation often." It is to increase closure at the dimensions whose ambiguity becomes expensive at that boundary.

This also clarifies the role of humans in the loop. Human review is not a generic safety ornament. It is one possible closure operator, especially valuable when the criterion is tacit, the situation exceeds the represented state, or responsibility attaches to endorsement. Conversely, a human should not

be required to manually re-check properties that a schema, invariant, or deterministic test can close more reliably. The division of labor should follow the kind of uncertainty, not a ritual preference for either automation or human control.

The hypothesis is experimentally tractable. One can hold an underlying task constant and vary its transition from read-only inspection to proposed mutation to committed mutation. At each boundary, measure which additional distinctions must be settled for reliable performance. A stronger version would annotate each transition with four fields: OPEN DIMENSIONS, CLOSURE OPERATOR, REVERSIBILITY, and REPAIR PATH. Systems could then be compared not by how “natural” their language appears, but by how well their closure topology matches the consequences they are permitted to cause.

9 Implications

This wager recasts several familiar debates.

First, the question “are prompts programs?” becomes less important. A prompt can be one source of constraints inside a larger program whose closure boundaries also include schemas, world state, tool contracts, demonstrations, tests, review gates, and temporal ledgers. Calling the prompt either “code” or “not code” misses the distributed operational object.

Second, natural-language programmability no longer requires natural language to acquire a complete compositional semantics equivalent to a conventional programming language. A system can be programmable if the architecture knows where ordinary-language ambiguity may persist and where another representation must close it before consequence. This is a different route to reliability from making all user language formal in advance.

Third, failure becomes part of specification only when it changes future closure. A bad output does not automatically teach the system anything. It becomes programmatically significant when the mismatch is diagnosed, represented, and made binding as a future test, rule, example, or changed policy.

Finally, the distinction opens a further question about authorship and responsibility that this paper does not yet settle. If sentence production is distributed but selection and publication are closure points, then attribution may track authority over certain settlements more closely than token generation. The present evidence is insufficient to make that a strong scholarly claim; it belongs in the next forage rather than being smuggled into this one.

10 Limitations and unresolved territory

Local closure is a comparative concept introduced here, not a source term and not yet a formal calculus. The paper does not provide a universal measure of ambiguity, prove that every agent architecture decomposes cleanly into closure events, or establish direct historical influence among SHRDLU, Attempto, programming by demonstration, situated-action research, and contemporary language-model systems. The histories intersect unevenly and often solve different problems.

The argument also risks making “closure” so broad that any decision counts. The remedy is

operational specificity: a closure claim should identify the previously open alternatives, the representation or actor that settles them, the downstream state transition that requires settlement, and the conditions under which the settlement can be reopened or repaired. If those cannot be named, the concept has not yet explained anything.

A final limitation is normative. Consequence-proportional closure sounds sensible but can be miscalibrated. Safety gates can incorrectly hold actions whose risks have already been resolved (Serdar and Mertayak 2026). Metrics can stabilize the wrong target. Human review can become ceremony. More closure can therefore reduce capability or preserve stale constraints. The principle requires calibration, not merely additional gates.

11 Conclusion

The deepest change in natural-language programming is not that words have become executable or that formalization has been postponed. Those claims are historically too easy to break. The more consequential development is an architecture in which different parts of a specification can remain underdetermined until different consequences require them to settle.

That architecture makes the program difficult to point to because it is distributed across time and representation. Some of it is in language. Some is in the world model. Some is in schemas and permissions. Some appears temporarily as code. Some is learned from failure. Some is held in the live judgment of a person who can still say no. Some determines whether a proposed action ever reaches the world. Some records that an old rule no longer binds.

The program, on this account, is where ambiguity stops—not everywhere, and not once, but locally, repeatedly, and in an order that determines what the system can safely make consequential. The research task is no longer to ask whether natural language is formal enough to program. It is to discover which ambiguities can remain alive, which must close, who or what has authority to close them, and how close to consequence the system can safely wait.

References

- Austin, J. L. (1962). *How to Do Things with Words*. Ed. by J. O. Urmson. Oxford: Clarendon Press.
- Brooks, Rodney A. (1991). “Intelligence without Representation”. In: *Artificial Intelligence* 47.1–3, pp. 139–159.
- Clark, Herbert H. and Deanna Wilkes-Gibbs (1986). “Referring as a Collaborative Process”. In: *Cognition* 22.1, pp. 1–39. DOI: 10.1016/0010-0277(86)90010-7.
- Fuchs, Norbert E. and Rolf Schwitter (1996). *Attempto Controlled English (ACE)*. arXiv: cmp-lg/9603003. URL: <https://arxiv.org/abs/cmp-lg/9603003>.
- Naur, Peter (1985). “Programming as Theory Building”. In: *Microprocessing and Microprogramming* 15, pp. 253–261.
- OpenAI (2026a). *Programmatic Tool Calling*. API documentation; accessed 2026-08-17. URL: <https://developers.openai.com/api/docs/guides/tools-programmatic-tool-calling>.
- (2026b). *Structured Model Outputs*. API documentation; accessed 2026-08-17. URL: <https://developers.openai.com/api/docs/guides/structured-outputs>.
- Pan, Linyue et al. (2026). *Natural-Language Agent Harnesses*. arXiv: 2603.25723.

- Pei, Zehua et al. (2026). *SCOPE: Prompt Evolution for Enhancing Agent Effectiveness*. arXiv: 2512.15374.
- Serdar, Oguz and Cuneyt Mertayak (2026). *SteerBench-Work: A Benchmark for Agent Steering at Action Boundaries*. arXiv: 2608.12654.
- Shi, Junjie et al. (2026). *REprompt: Prompt Generation for Intelligent Software Development Guided by Requirements Engineering*. arXiv: 2601.16507.
- Smith, David Canfield (1993). “Pygmalion: An Executable Electronic Blackboard”. In: *Watch What I Do: Programming by Demonstration*. Ed. by Allen Cypher. MIT Press. URL: <https://acypher.com/wwid/Chapters/01Pygmalion.html>.
- Suchman, Lucy A. (Feb. 1985). *Plans and Situated Actions: The Problem of Human-Machine Communication*. Tech. rep. ISL-6. Palo Alto, CA: Xerox Palo Alto Research Center.
- Ta, Anh, Junjie Zhu, and Shahin Shayandeh (2026). *Reinforced Agent: Inference-Time Feedback for Tool-Calling Agents*. arXiv: 2604.27233.
- Winograd, Terry (1973). “A Procedural Model of Language Understanding”. In: *Computer Models of Thought and Language*. Ed. by Roger C. Schank and Kenneth M. Colby. San Francisco: W. H. Freeman, pp. 152–186.
- Zhu, Haoyuan (2026). *Dead text or binding clause? Measuring and restoring constraint influence in black-box LLM dialogues*. arXiv: 2608.12599.

Appendix A: Assembly instrument

This working paper belongs to a portable research field. The exact assembly instrument is preserved as `_PROMPTS/10__slipcase-v15.55-AM__assembly.poml`. Assembly prompt SHA-256: `a5ef0e2e3ea56463e25065f19e7313b4e9c8c569b8cceb1a3c5d9f05bd83598a`.

Appendix B: Making history

The prior field’s 40 immutable zettel payloads were preserved without rewriting. The earlier “restriction migration” paper was treated as a prior interpretation, not evidence. This paper was derived by following the field’s DEEPER QUESTION, TENSION, MISSING, BOUNDARY, and TEST edges. No new external web research was introduced in this compile. Full making history is preserved in `the-program-is-where-ambiguity-stops__MAKING_HISTORY.txt`.

Appendix C: Replication path

The companion `index.html` is a standalone replication capsule containing the evidence deck, graph state, bibliography, exact assembly prompt, paper, source map, making history, and rebuild instructions. See `000__RETURN_PATH.txt` for the checkpoint identity and rejoin phrase.